

Production LLM Operations (LLMOps)

Failure Classes · Deployment Topologies · Metrics Stack · Continuous Evaluation · Drift Detection · Guardrails · Incident Response

WEEK 4 · DAY 22

Monday, July 27 2026

Day 22 of 84 — Week 4 Begins

THE CORE PROBLEM OF LLMOps: a normal service failure is loud — a 500, a stack trace, an alert. An LLM system failure is a 200 OK containing a well-formed, plausible, subtly WRONG answer. No exception. No alert. Your failure mode is a successful HTTP response.

Sec	Topic	Coverage
Sec 1	Four Failure Classes	Infrastructure / Quality / Cost / Safety
Sec 2	Deployment Topologies	Managed / single-node / pooled / hybrid+fallback
Sec 3	LLM Metrics Stack	RED + token economics + quality + agent health
Sec 4	Continuous Evaluation	CanarySuite, ContinuousEvaluator, slow-decay trend
Sec 5	Drift Detection	Input drift / corpus drift / model drift + triage matrix
Sec 6	Guardrails	5 runtime gates on generated SQL; defense in depth
Sec 7	Incident Response	Triage runbook, mitigate-before-diagnose, alert design
Sec 8	FAANG LLMOps	Google/Microsoft/Amazon/Meta/Netflix operational practice
Sec 9	Architect's Lens	Maturity ladder L0-L4, eval budget, version everything
Sec 10	Exercises + Summary	Canary build, drift triggers, gauntlet, incident drill

Section 1 — The Four Failure Classes

Every LLM production incident falls into one of four buckets. Your monitoring must cover all four, because each is detected differently — and three of the four are invisible to standard observability.

```
CLASS 1 – INFRASTRUCTURE FAILURE    (loud, familiar)
  GPU OOM, vLLM crash, model server unreachable, timeout, 5xx
  Detection:      standard APM – error rate, latency, health checks
  Time to detect: seconds
  This is the ONLY class normal monitoring catches.

CLASS 2 – QUALITY DEGRADATION        (silent, gradual)
  Accuracy drops 84% -> 71% over three weeks. No errors. No alerts.
  Causes:  input distribution shift, RAG index staleness, prompt edit,
           model/provider version change, retrieval corpus drift
  Detection:      continuous eval on a canary set + user feedback
  Time to detect without eval: WEEKS. Usually a user reports it.

CLASS 3 – COST BLOWOUT                (silent, fast)
  Token spend triples overnight. Nothing is "broken."
  Causes:  retry storm, prompt bloat from a schema change, agent loop
           not terminating, a new tenant with 40k-token schemas
  Detection:      token-per-request histogram + spend rate alerts
  Time to detect without metrics: the monthly invoice.

CLASS 4 – SAFETY / CORRECTNESS        (silent, high blast radius)
  Hallucinated SQL executes. PII leaks into a prompt. Tenant A retrieves
  tenant B's schema. Prompt injection succeeds.
  Detection:      guardrail counters, faithfulness checks, isolation tests
  Time to detect without controls: until someone external notices.

ARCHITECT'S RULE:
  Classes 2-4 are invisible to standard observability.
  If your LLM monitoring looks like your microservice monitoring,
  you are monitoring one of four failure classes.
```

Section 2 — Deployment Topologies

Topology	Shape	Use When
1. Managed API	Client → your API → provider API	< 200 q/hr, no data-residency constraint, quality ceiling > unit cost
2. Self-hosted single-node	Client → your API → vLLM on one GPU box	Steady predictable load, one model, internal tool
3. Self-hosted pooled	LB → API replicas → 70B pool + 8B pool	> 500 q/hr sustained, mixed model sizes, cost matters (the capstone)
4. Hybrid + fallback	Primary self-hosted, managed API on saturation	Strict availability SLO with spiky traffic

The hidden cost of Topology 4 is behavioral alignment: two prompt paths, two cost models, and two sets of eval results to keep in agreement.

```
class Backend(Enum):
    PRIMARY = "self_hosted_vllm"
    FALLBACK = "managed_api"

@dataclass
class RoutingPolicy:
    queue_depth_threshold: int = 40      # route to fallback above this
    p95_latency_threshold: float = 6000.0 # ms
    fallback_traffic_cap: float = 0.25    # never send >25% to fallback
    circuit_open_secs: int = 30           # how long to stay tripped

class BackendRouter:
    """Routes between backends. Implements a circuit breaker on the primary."""

    def _circuit_is_open(self) -> bool:
        return time.time() < self._circuit_open_until

    def trip_circuit(self, reason: str):
        self._circuit_open_until = time.time() + self.policy.circuit_open_secs
        # emit metric: llm_circuit_trips_total{reason=...}

    def choose(self, queue_depth: int, recent_p95_ms: float) -> Backend:
        self._total_count += 1
        # HARD CAP: never exceed the fallback traffic budget.
        # Without this, one bad minute sends 100% of traffic to the
        # expensive backend and the cost incident replaces the latency one.
        if self._fallback_count / self._total_count >= self.policy.fallback_traffic_cap:
            return Backend.PRIMARY
        if self._circuit_is_open():
            self._fallback_count += 1; return Backend.FALLBACK
        if queue_depth > self.policy.queue_depth_threshold:
            self.trip_circuit("queue_depth")
            self._fallback_count += 1; return Backend.FALLBACK
        if recent_p95_ms > self.policy.p95_latency_threshold:
            self.trip_circuit("latency")
            self._fallback_count += 1; return Backend.FALLBACK
        return Backend.PRIMARY
```

Section 3 — The LLM Metrics Stack

Standard RED metrics cover Class 1 only. LLM systems need four metric families. A dashboard showing only latency is a web-service dashboard.

[illegible]

Section 4 — Continuous Evaluation (Detecting Class 2)

Quality degradation is invisible without a measurement that runs on a schedule. This is the single highest-value component in LLMOps — it is what converts silent quality decay into a page.

```
@dataclass
class CanarySuite:
    """A small, fixed, HIGH-SIGNAL eval set. NOT the full benchmark.
    Sized to run every hour without meaningful cost."""
    name: str
    cases: list # list[EvalCase] from Day 21
    baseline: float # accuracy at the last blessed release
    tolerance_pp: float = 3.0 # alert if we drop more than this (pp)
    min_cases: int = 40 # below this, noise swamps signal

    def is_significant(self, observed: float) -> bool:
        """A drop within tolerance is noise, not a regression."""
        return (self.baseline - observed) * 100 > self.tolerance_pp

class ContinuousEvaluator:
    """Runs canary suites on a schedule against the LIVE production system –
    not a test double. The whole point is to measure reality."""

    async def run_suite(self, suite: CanarySuite) -> CanaryRun:
        report = await CapstoneEvaluator(self.system).run(suite.cases) # Day 21
        run = CanaryRun(suite=suite.name, timestamp=datetime.utcnow(),
            accuracy=report.execution_accuracy,
            p95_latency=report.p95_latency,
            failures=report.failures)
        self.history.setdefault(suite.name, []).append(run)
        canary_accuracy.labels(suite=suite.name, difficulty="all").set(run.accuracy)

        if suite.is_significant(run.accuracy):
            await self.alerter.fire(
                severity="page", title=f"Canary regression: {suite.name}",
                detail=(f"accuracy {run.accuracy:.1%} vs baseline {suite.baseline:.1%} "
                    f"(-{(suite.baseline-run.accuracy)*100:.1f}pp)",
                runbook="runbooks/canary-regression.md",
                context={"new_failures": run.failures[:5]})
            return run

    def trend(self, suite_name: str, window: int = 24) -> dict:
        """Detect SLOW decay that never trips the single-run threshold.
        A 0.5pp drop per day never alerts – but it is 15pp in a month.
        This is the most commonly missed regression."""
        runs = self.history.get(suite_name, [])[-window:]
        if len(runs) < 6: return {"status": "insufficient_data", "n": len(runs)}
        accs = [r.accuracy for r in runs]
        drift = (statistics.mean(accs[-3:]) - statistics.mean(accs[:3])) * 100
        return {"status": "degrading" if drift < -1.5 else "stable",
            "drift_pp": round(drift, 2), "n_runs": len(runs)}

    async def schedule(self, interval_secs: int = 3600):
        """Run every suite on an interval. Deploy as a sidecar or cron job."""
        while True:
            for suite in self.suites:
```

```
try:    await self.run_suite(suite)
except Exception as e:
    await self.alerter.fire(severity="warn",
                           title=f"Canary run failed: {suite.name}", detail=str(e))
await asyncio.sleep(interval_secs)
```

■■ CANARY SUITE DESIGN – build THREE suites, not one ████████████████████████

- # golden_core (60 cases) easy/medium, must NEVER regress.
- # tolerance 2pp. Regression = PAGE.
- # golden_hard (40 cases) hard/extra, expected noisier.
- # tolerance 5pp. Regression = TICKET.
- # production_replay (50 cases) sampled from REAL traffic, gold answers
- # curated by a human.
- #
- # The third suite is the one teams skip and the one that matters most:
- # benchmark distribution != your distribution.

Section 5 — Drift Detection

Continuous eval tells you quality dropped. Drift detection tells you WHY — and often warns you before quality drops. Three distinct things drift, with different detectors and different fixes.

```
# ■■■ DRIFT 1: INPUT DRIFT – the questions users ask changed ■■■■■■■■■■■■■■■■■■■■
```

```
class InputDriftDetector:
```

```
    def check(self, recent_questions: list[str]) -> dict:
```

```
        vecs = self.embedder.encode(recent_questions, normalize_embeddings=True)
```

```
        cur_centroid = vecs.mean(axis=0)
```

```
        # Cosine distance between distribution centroids
```

```
        centroid_shift = 1.0 - float(np.dot(self.ref_centroid, cur_centroid) /
```

```
            (np.linalg.norm(self.ref_centroid)*np.linalg.norm(cur_centroid)))
```

```
        # Kolmogorov-Smirnov test on question length distribution
```

```
        cur_lengths = [len(q.split()) for q in recent_questions]
```

```
        ks_stat, p_value = stats.ks_2samp(self.ref_lengths, cur_lengths)
```

```
        drifted = centroid_shift > 0.15 or p_value < 0.01
```

```
        return {"drifted":drifted, "centroid_shift":round(centroid_shift,4),
```

```
                "length_ks_p":round(float(p_value),6),
```

```
                "interpretation":"Users ask materially different questions."
```

```
                    "Refresh few-shot examples; re-check retrieval recall."}
```

```
# ■■■ DRIFT 2: CORPUS DRIFT – the retrieved data changed ■■■■■■■■■■■■■■■■■■■■
```

```
class CorpusDriftDetector:
```

```
    """The most UNDER-MONITORED failure in RAG systems. A schema migration
```

```
    silently invalidates the index; retrieval keeps returning confident
```

```
    results for tables that no longer exist."""
```

```
    async def check(self, schema_name: str, tenant_id: str) -> dict:
```

```
        live_fp      = await self.db.schema_fingerprint(schema_name)          # Day 16
```

```
        indexed_fp   = await self.indexer.stored_fingerprint(schema_name, tenant_id)
```

```
        live_tables  = set(await self.db.list_tables(schema_name))
```

```
        indexed_tables = set(await self.indexer.list_indexed_tables(
```

```
                                schema_name, tenant_id))
```

```
        stale       = indexed_tables - live_tables           # indexed but DROPPED: DANGEROUS
```

```
        missing     = live_tables - indexed_tables         # exists but not indexed: invisible
```

```
        return {"in_sync": live_fp == indexed_fp,
```

```
                "stale_tables": sorted(stale), "missing_tables": sorted(missing),
```

```
                "severity": "critical" if stale else "warn" if missing else "ok",
```

```
                "action": "Re-index immediately – retrieval is returning tables "
```

```
                            "that no longer exist in the database." if stale else
```

```
                            "Re-index – new tables are invisible to retrieval."}]
```

```
# ■■■ DRIFT 3: MODEL DRIFT – the model itself changed ■■■■■■■■■■■■■■■■■■■■
```

```
class ModelDriftDetector:
```

```
    """Same prompt, same input, different output than last week.
```

```
    On managed APIs this happens WITHOUT a version bump on your side."""
```

```
    FINGERPRINT_PROMPTS = [
```

```
        "Return only the number: what is 17 * 23?",
```

```
        "Return only valid JSON: {'status': 'ok'}",
```

```
        "Count the tables in: CREATE TABLE a(x int); CREATE TABLE b(y int);" ]
```

```
    async def fingerprint(self) -> list[str]:
```

```
        """Deterministic settings – ANY output change is a model change."""
```

```
        return [ (await self.client.complete(model=self.model, prompt=p,
```

```
                                                temperature=0.0, top_p=1.0, seed=42, max_tokens=32)).strip()
```

```
                  for p in self.FINGERPRINT_PROMPTS ]
```

```
    asnc def check(self, stored_fingerprint: list[str]) -> dict:
```

```
current = await self.fingerprint()
changed = [i for i,(a,b) in enumerate(zip(stored_fingerprint,current)) if a!=b]
return {"model_changed": bool(changed), "changed_idx": changed,
        "action": "Model behavior changed under a FIXED prompt. Re-run the "
                  "full benchmark – your baseline may no longer be valid."}

# ■■ DRIFT TRIAGE MATRIX – which detector fired tells you the fix ■■■■■■■■
# input drifted, corpus ok,   model ok      -> new query types
#                                     FIX: few-shot + retrieval
# input ok,      corpus drift, model ok     -> schema migration
#                                     FIX: re-index (fastest fix)
# input ok,      corpus ok,   model drifted -> provider/checkpoint change
#                                     FIX: pin version, re-baseline
# NOTHING drifted                                     -> look at YOUR last deploy.
#                                     A prompt edit is the usual cause.
```


Section 6 — Guardrails as Runtime Controls (Class 4)

A guardrail expressed as a prompt instruction is **ADVISORY** and can be argued away by a clever input. A guardrail applied as a separate enforcement pass around the model is a **CONTROL**. These gates run on generated SQL, not in the system prompt.

[illegible]

```
guardrail_blocks.labels(guard="tenant_isolation").inc()
raise GuardrailViolation("tenant_isolation",
    f"SQL references tables outside tenant scope: {sorted(rogue)}")

# ■■ Full gate chain – CHEAPEST FIRST ████████████████████████████████████████
async def apply(self, sql: str, allowed_tables: set[str]) -> str:
    self.assert_read_only(sql)           # pure AST – microseconds
    self.assert_no_pii(sql)
    self.assert_tenant_scope(sql, allowed_tables)
    sql = self.enforce_row_cap(sql)
    await self.assert_affordable(sql)     # hits the DB – most expensive, last
    return sql

# ENFORCEMENT LAYER RULE (from Day 21's design review):
# These app-layer gates are NECESSARY BUT NOT SUFFICIENT. The DB-level
# SELECT-only grant and statement_timeout are the controls that survive a
# bug in this file. Application guardrails and database grants are
# defense in depth – not alternatives.
```

Section 7 — Incident Response for LLM Systems

An LLM runbook differs from a service runbook in one way: the first question is not 'what broke' but 'is it actually broken?'

LLM INCIDENT TRIAGE – first 10 minutes

STEP 0 – CLASSIFY (30 seconds)

Is there an error signal (5xx, timeout, OOM)?

YES -> Class 1. Standard incident. Go to the infra runbook.

NO -> Class 2/3/4. Continue. This is the LLM-specific path.

STEP 1 – CONFIRM IT IS REAL (2 min)

Run the canary suite ON DEMAND. Do not trust a single user report.

within tolerance -> likely user- or tenant-specific, not systemic.

Scope down before escalating.

regressed -> systemic. Continue.

STEP 2 – LOCATE THE CHANGE (3 min) [-95% of causes, in this order]

a) Did WE deploy? (prompt, model version, retrieval config, code)

-> check the deploy log. A PROMPT EDIT is the most common cause.

b) Did the CORPUS change? -> run CorpusDriftDetector.

-> a schema migration is the second most common cause.

c) Did the MODEL change? -> run ModelDriftDetector fingerprint.

-> on managed APIs this happens without any action from you.

STEP 3 – MITIGATE BEFORE DIAGNOSING (2 min)

Fastest first:

- Roll back the prompt/config (seconds – prompts are versioned data)

- Re-index the corpus (minutes – fixes corpus drift)

- Pin the model version (minutes – fixes provider drift)

- Route to the fallback backend (seconds – Topology 4)

- Disable the failing feature path (seconds – degrade, do not fail)

DO NOT debug a live regression. Mitigate, then diagnose off the clock.

STEP 4 – CAPTURE FOR THE EVAL SET (ongoing)

Every confirmed incident becomes a canary case. An incident that cannot recur silently is worth more than the incident cost. This is the ONLY mechanism that makes the system monotonically safer.

```
class IncidentRecorder:
```

```
    async def close(self, incident: Incident, new_cases: list):
```

```
        if not new_cases:
```

```
            raise ValueError(
```

```
                "Cannot close an LLM incident with zero new eval cases. "
```

```
                "Without a regression test, this incident is scheduled to recur.")
```

```
        incident.eval_cases_added = len(new_cases)
```

```
        await self.canary_store.append("production_replay", new_cases)
```

```
        await self.rebaseline("production_replay")
```

ALERT DESIGN – what pages, what tickets, what is dashboard-only

PAGE (wake someone up)

- Error rate > 2% for 5 min (Class 1)

- golden_core canary accuracy drops > 2pp (Class 2)

- Guardrail block rate > 1% of requests (Class 4)

- Tenant isolation violation – ANY, count 1 (Class 4)

- Spend rate > 3x the 7-day trailing average (Class 3)

TICKET (next business day)

- golden_hard canary drops > 5pp (Class 2)
- Corpus drift: missing_tables non-empty (Class 2)
- p99 latency > SLO for 1 hour (Class 1)
- Validation retry rate > 25% (Class 2)

DASHBOARD ONLY (context, not signal)

- Per-phase latency breakdown | token distribution by tenant
- Agent step histogram | exact-match rate (informational per Day 21)

ANTI-PATTERN: alerting on exact-match rate. It is not a correctness signal (Day 21), so it generates pages nobody can action.

Section 8 — FAANG LLMOps Practice

Google — SRE Discipline Applied to ML Systems

Google's SRE writing established error budgets and SLO-driven operations, and Google's ML-specific guidance (the 'Rules of Machine Learning' and the 'ML Test Score' work) extends that discipline to models: treat training data, features, and model behavior as production dependencies needing their own tests and monitoring — not artifacts validated once before launch. The transferable practice: a model or prompt is a deployed dependency with a version, an owner, and a rollback path. Google's framing that ML systems have all the maintenance problems of normal code PLUS an additional set is the cleanest justification for why Week 4 exists.

Microsoft — Azure AI Evaluation and Content Safety as Separate Services

Microsoft ships evaluation and safety as first-class platform services rather than things each team rebuilds. Azure AI Foundry provides evaluators for groundedness, relevance, and coherence, runnable offline against a dataset and online against production traffic; Azure AI Content Safety provides moderation and prompt-injection detection (Prompt Shields) callable in the request path. The architectural lesson is SEPARATION: content safety (Class 4) and quality evaluation (Class 2) are distinct systems with distinct interfaces, triggered by different signals and owned by different people. Teams that merge them into one 'quality service' end up unable to page correctly on either.

Amazon — Bedrock Guardrails as a Separate Enforcement Layer

Bedrock Guardrails is configured independently of the model and applied to both the input and the output of a request, supporting denied topics, content filters, PII detection and redaction, and contextual grounding checks. The design decision worth copying: the guardrail is NOT part of the prompt. A guardrail expressed as prompt instructions is advisory and can be argued away by a sufficiently clever input; a guardrail applied as a separate pass around the model is a control. The SQLGuardrails class in Section 6 follows the same principle — it runs on the generated SQL as an independent gate, not as a line asking the model to behave.

Meta — Llama Guard and Purple Llama

Meta released Llama Guard as an openly available safeguard model — a classifier evaluating prompts and responses against a taxonomy of hazards — under the Purple Llama umbrella of trust-and-safety tooling, alongside CyberSecEval for security benchmarking. The pattern this establishes is the SMALL DEDICATED SAFETY MODEL: rather than asking your large generation model to police itself, run a cheap specialized classifier alongside it. This is Day 20's mixed model sizing applied to safety instead of retrieval — a 7B guard reviewing a 70B generator's output is far cheaper than the generation call it protects, so the safety pass is essentially free relative to the request it guards.

Netflix — Automated Controls at the Point of Execution

Netflix's data platform engineering writing consistently emphasizes that at their scale the expensive failure is not an error but an UNBOUNDED QUERY — which is why plan inspection and cost controls sit in the execution path rather than in a review process. The principle generalizes directly: when a machine generates the work, the control must be automated at the point of execution, because there is no human in the loop to catch the pathological case. Every guardrail in Section 6 exists for this reason. A human writing SQL by hand will hesitate before joining two petabyte tables; a model will not hesitate, so the system must.

Section 9 — Architect's Lens

THE LLMOPS MATURITY LADDER – where most teams actually are

- LEVEL 0 "It's deployed."
Health check + error rate. Detects Class 1 only.
-> Quality regressions found by users. Cost found by invoice.
MOST TEAMS SHIPPING LLM FEATURES TODAY ARE HERE.
- LEVEL 1 "We log everything."
Full request/response logging, trace_id, token counts.
-> Can now DIAGNOSE an incident someone else reported.
Still cannot DETECT one.
- LEVEL 2 "We have a canary suite."
Scheduled eval, baseline, regression alert.
-> Detects Class 2. The single biggest jump on the ladder.
Everything below is table stakes; LLMops starts here.
- LEVEL 3 "We have guardrails and cost controls."
Runtime enforcement, spend alerts, drift detectors.
-> Detects Classes 3 and 4 before they become incidents.
- LEVEL 4 "Incidents become eval cases automatically."
Feedback loop closed. System gets monotonically safer.
-> Regression is structurally impossible for any past incident.

THE JUMP FROM 1 -> 2 IS THE ONE THAT MATTERS.

It is also the cheapest: 60 curated cases and an hourly cron job.

BUDGET REALITY – what continuous eval actually costs

golden_core: 60 cases x hourly = 1,440 evals/day

At \$0.0063/query (Day 21) = \$9.07/day = ~\$272/month

Compare: ONE undetected 3-week quality regression on a customer-facing feature. The eval budget is not the expensive line item.

WHAT TO INSTRUMENT FIRST IF YOU CAN ONLY DO THREE THINGS

1. Canary suite + regression alert -> Class 2 (the invisible one)
 2. Token histogram + spend rate alert -> Class 3 (the fast one)
 3. Guardrail block counter -> Class 4 (the dangerous one)
- Class 1 is already covered by whatever APM you have. Do not start there.

THE QUESTION THAT EXPOSES LEVEL 0

"If your model's quality dropped 15% right now, how long until you knew, and what would tell you?"

If the answer names a PERSON rather than a SYSTEM, you are at Level 0 or 1.

VERSION EVERYTHING THAT AFFECTS OUTPUT

Prompts, few-shot example banks, retrieval configs, model checkpoints, and embedding models are all PRODUCTION ARTIFACTS.

A prompt edit with no version and no eval run is an UNMEASURED DEPLOY. Day 21 flagged the golden-set CI gate as the #1 open item for exactly this reason. Tomorrow (Day 23) closes it.

Section 10 — Exercises and Summary

Exercise 1: Build the Canary Suite

```
# Curate 60 cases from your Day 21 SPIDER eval set into golden_core.
# Requirement: easy/medium only, and they must ALL pass today.
# Record the baseline accuracy. Set tolerance to 2pp.
# Run it TWICE with no changes – confirm run-to-run variance < tolerance.
# If variance exceeds tolerance, your suite is too small or temperature > 0.
```

Exercise 2: Trigger Every Drift Detector

```
# Input drift: feed 50 questions from a different domain; confirm detection
# Corpus drift: DROP a table from the DB without re-indexing.
# Confirm severity=="critical" and the stale table is NAMED.
# Model drift: change the model name in config; confirm the fingerprint
# changes and model_changed==True.
```

Exercise 3: Guardrail Gauntlet

```
# Six adversarial inputs, one per gate:
# 1. A question phrased to induce DELETE -> assert_read_only blocks
# 2. SELECT * on the largest table, no LIMIT -> enforce_row_cap injects
# 3. A cross join of two large tables -> assert_affordable blocks
# 4. A question targeting an email column -> assert_no_pii blocks
# 5. A question naming another tenant's table -> assert_tenant_scope blocks
# 6. A benign aggregate query -> passes ALL gates unchanged
# Case 6 is the important one: guardrails that break normal queries get
# disabled by the team within a week.
```

Exercise 4: Run the Incident Drill

```
# Deliberately regress the system: edit the system prompt to remove the
# CoT instruction (Day 18). Then run triage in order and time yourself.
# - Did the canary catch it before you looked?
# - How long from canary alert to correct root cause?
# - Did you MITIGATE (revert) before diagnosing?
# Target: < 10 minutes to correct root cause using only the runbook.
```

Summary

Concept	Key Point
Core LLMOps problem	Failure mode is a 200 OK containing a wrong answer
Four failure classes	Infrastructure / Quality / Cost / Safety — detected differently
Class 1 only	What standard APM catches; 3 of 4 classes invisible to it
Topology 1-4	Managed API / single-node / pooled / hybrid with fallback
BackendRouter	Circuit breaker + fallback traffic cap (never >25%)
Four metric families	RED + token economics + quality signals + agent-loop health
Four wall numbers	p95 latency, canary accuracy, cost/query, guardrail block rate
ContinuousEvaluator	Scheduled canary against LIVE production, not a test double

CanarySuite	baseline + tolerance_pp; is_significant() filters noise
Three suites	golden_core (page) / golden_hard (ticket) / production_replay
production_replay	The suite teams skip; benchmark distribution ≠ yours
trend()	Catches slow decay that never trips a single-run threshold
Input drift	Embedding centroid shift + KS test on length distribution
Corpus drift	Fingerprint mismatch; stale_tables is critical severity
Model drift	Fixed prompts, temp=0, seed=42 — output change = model change
Drift triage matrix	Which detector fired tells you which fix applies
SQLGuardrails	5 gates: read-only, PII, tenant scope, row cap, plan cost
Gate ordering	Cheapest AST checks first; DB round trips last
Guardrail ≠ prompt	A prompt instruction is advisory; a separate pass is a control
Incident triage	Confirm real → locate change → mitigate → capture eval case
Mitigate before diagnose	Never debug a live regression
eval_cases_added	Close an incident with zero new cases and it WILL recur
Maturity ladder	Level 0→4; the 1→2 jump (canary suite) is the one that matters
Eval budget	~\$272/month for hourly 60-case canary — not the expensive line

NEXT: Day 23 — Prompt and Model Versioning

Treating prompts as versioned production artifacts, the golden-set CI gate flagged as Day 21's #1 open item, A/B testing prompt changes with statistical rigor, and safe rollout patterns for model upgrades.